

Exercícios Capítulo 3 - Patterson 4ª Edição

Exercício 3.1

O livro mostra como adicionar e subtrair números binários e decimais. Entretanto, outros sistemas de numeração eram igualmente muito populares ao tratar os computadores. (Base 8) o sistema de numeração Octal era um destes. A tabela a seguir mostra dois pares de números em octal.

	A	B
a.	3174	0522
b.	4165	1654

3.1.1 Qual é a soma de A e B se eles são números octal de 12 bits sem sinal? O resultado deve ser escrito em octal. Mostre seu trabalho.

a.	3716
b.	6041

3.1.2 Qual é a soma de A e B se eles representam números em octal com 12 bits armazenados no formato do sinal-magnitude? O resultado deve ser escrito em octal. Mostre seu trabalho.

a.	3716
b.	1467

3.1.3 Converta A em um número decimal, supor que é sem sinal. Repita a conversão supondo que o formato é sinal-magnitude. Mostre seu trabalho.

a.	1660	1660
b.	2165	-117

Considere os pares de números octais abaixo.

	A	B
a.	7040	0444
b.	4365	3412

3.1.4 Qual o resultado de A - B se eles representam números octais de 12 bits sem sinal ? Escreva o resultado em octal. Mostre seu trabalho.

a.	6374
b.	753

3.1.5 Qual o resultado de A - B se eles representam números octais de 12 bits em sinal-magnitude ? Escreva o resultado em octal. Mostre seu trabalho.

a.	7504 (-3504)
b.	7777 (-3777)

3.1.6 Converta A em um número binário. O que faz a base 8 (octal) um sistema de numeração atrativo para representar valores nos computadores?

a.	111000100000
b.	100011110101

Exercício 3.2

O Hexadecimal (base 16) é igualmente um sistema de numeração de uso geral para representar valores nos computadores. De fato, tornou-se muito mais popular do que octal. A seguinte tabela mostra pares de números em hexadecimal.

	A	B
a.	1446	672F
b.	2460	4935

3.2.1 Qual é a soma de A e B se eles representam números hexadecimais de 16 bits sem sinal? O resultado deve ser escrito no hexadecimal. Mostre seu trabalho.

a.	7B75
b.	6D95

3.2.2 Qual é a soma de A e B se eles representam números hexadecimais de 16 bits em sinal-magnitude ? O resultado deve ser escrito no hexadecimal. Mostre seu trabalho.

a.	7B75
b.	6D95

3.2.3 Converta A em um número decimal, supondo que é sem sinal. Repita a suposição que armazenou no formato do sinal-magnitude. Mostre seu trabalho.

a.	5190	5190
b.	9312	9312

Dados os pares de números hexadecimais abaixo

	A	B
a.	C352	36AE
b.	5ED4	07A4

3.2.4 Qual o valor de A - B sendo que eles representam números hexadecimais de 16 bits sem sinal? O resultado deve ser escrito no hexadecimal. Mostre seu trabalho.

a.	8CA4
b.	5730

3.2.5 Qual o valor de A - B sendo que eles representam números hexadecimais de 16 bits em sinal-magnitude? O resultado deve ser escrito no hexadecimal. Mostre seu trabalho.

a.	FA00
b.	5730

3.2.6 Converta A em um número binário. O que torna a base 16 (hexadecimal) um sistema de numeração atrativo para representar valores nos computadores?

a.	1100001101010010
b.	0101111011010100

Exercício 3.3

O *overflow* ocorre quando um resultado é demasiado grande ser representado dado exatamente um tamanho finito da palavra. O *underflow* ocorre quando um número é demasiado pequeno para ser armazenado corretamente. Considere os pares de números decimais abaixo.

	A	B
a.	216	255
b.	185	122

3.3.1 Supor que A e B são inteiros decimais representados em 8 bits sem sinal. Calcule $A - B$. Ocorre *overflow*, *underflow*, ou umas nenhuma?

a.	Underflow (−39)
b.	Neither (63)

3.3.2 Supor que A e B são inteiros decimais representados em 8 bits no formato sinal magnitude. Calcule um $A + B$. Ocorre *overflow*, *underflow*, ou umas nenhuma?

a.	Overflow (result = −215, which does not fit into an SM 8-bit format)
b.	Neither (65)

3.3.3 Supor que A e B são inteiros decimais representados em 8 bits no formato sinal magnitude. Calcule um $A - B$. Ocorre *overflow*, *underflow*, ou umas nenhuma?

a.	Neither (39)
b.	Overflow (result = −179, which does not fit into an SM 8-bit format)

Exercício 3.4

Vamos ver com mais detalhes a multiplicação. Nós vamos usar os números na tabela a seguir.

	A	B
a.	62	12
b.	35	26

3.4.1 Usando uma tabela semelhante à mostrada na Figura 3.7, calcule o produto dos números inteiros A e B octais de 6 bits sem sinal usando o hardware descrito na Figura 3.4. Você deve mostrar o conteúdo de cada registro em cada etapa.

a. 62×12

Step	Action	Multiplier	Multiplicand	Product
0	Initial Vals	001 010	000 000 110 010	000 000 000 000
1	Isb = 0, no op	001 010	000 000 110 010	000 000 000 000
	Lshift Mcand	001 010	000 001 100 100	000 000 000 000
	Rshift Mplier	000 101	000 001 100 100	000 000 000 000
2	Prod = Prod + Mcand	000 101	000 001 100 100	000 001 100 100
	Lshift Mcand	000 101	000 011 001 000	000 001 100 100
	Rshift Mplier	000 010	000 011 001 000	000 001 100 100
3	Isb = 0, no op	000 010	000 011 001 000	000 001 100 100
	Lshift Mcand	000 010	000 110 010 000	000 001 100 100
	Rshift Mplier	000 001	000 110 010 000	000 001 100 100
4	Prod = Prod + Mcand	000 001	000 110 010 000	000 111 110 100
	Lshift Mcand	000 001	001 100 100 000	000 111 110 100
	Rshift Mplier	000 000	001 100 100 000	000 111 110 100
5	Isb = 0, no op	000 000	001 100 100 000	000 111 110 100
	Lshift Mcand	000 000	011 001 000 000	000 111 110 100
	Rshift Mplier	000 000	011 001 000 000	000 111 110 100
6	Isb = 0, no op	000 000	110 010 000 000	000 111 110 100
	Lshift Mcand	000 000	110 010 000 000	000 111 110 100
	Rshift Mplier	000 000	110 010 000 000	000 111 110 100

b. 35×26

Step	Action	Multiplier	Multiplicand	Product
0	Initial Vals	010 110	000 000 011 101	000 000 000 000
1	Isb = 0, no op	010 110	000 000 011 101	000 000 000 000
	Lshift Mcand	010 110	000 000 111 010	000 000 000 000
	Rshift Mplier	001 011	000 000 111 010	000 000 000 000
2	Prod = Prod + Mcand	001 011	000 000 111 010	000 000 111 010
	Lshift Mcand	001 011	000 001 110 100	000 000 111 010
	Rshift Mplier	000 101	000 001 110 100	000 000 111 010

Step	Action	Multiplier	Multiplicand	Product
3	Prod = Prod + Mcand	000 101	000 001 110 100	000 010 101 110
	Lshift Mcand	000 101	000 011 101 000	000 010 101 110
	Rshift Mplier	000 010	000 011 101 000	000 010 101 110
4	Isb = 0, no op	000 010	000 011 101 000	000 010 101 110
	Lshift Mcand	000 010	000 111 010 000	000 010 101 110
	Rshift Mplier	000 001	000 111 010 000	000 010 101 110
5	Prod = Prod + Mcand	000 001	000 111 010 000	001 001 111 110
	Lshift Mcand	000 001	001 110 100 000	001 001 111 110
	Rshift Mplier	000 000	001 110 100 000	001 001 111 110
6	Isb = 0, no op	000 000	001 110 100 000	001 001 111 110
	Lshift Mcand	000 000	011 101 000 000	001 001 111 110
	Rshift Mplier	000 000	011 101 000 000	001 001 111 110

3.4.2 Usando a implementação otimizada em hardware calcule o produto dos números inteiros A e B de 6 bits octais sem sinal. Você deve mostrar o conteúdo de cada registro em cada etapa.

a. 62×12

Step	Action	Multiplicand	Product/Multiplier
0	Initial Vals	110 010	000 000 001 010
1	Isb = 0, no op	110 010	000 000 001 010
	Rshift Product	110 010	000 000 000 101
2	Prod = Prod + Mcand	110 010	110 010 000 101
	Rshift Mplier	110 010	011 001 000 010
3	Isb = 0, no op	110 010	011 001 000 010
	Rshift Mplier	110 010	001 100 100 001
4	Prod = Prod + Mcand	110 010	111 110 100 001
	Rshift Mplier	110 010	011 111 010 000
5	Isb = 0, no op	110 010	011 111 010 000
	Rshift Mplier	110 010	001 111 101 000
6	Isb = 0, no op	110 010	001 111 101 000
	Rshift Mplier	110 010	000 111 110 100

b. 35×26

Step	Action	Multiplicand	Product/Multiplier
0	Initial Vals	011 101	000 000 010 110
1	Isb = 0, no op	011 101	000 000 010 110
	Rshift Mplier	011 101	000 000 001 011
2	Prod = Prod + Mcand	011 101	011 101 001 011
	Rshift Product	011 101	001 110 100 101
3	Prod = Prod + Mcand	011 101	101 011 100 101
	Rshift Mplier	011 101	010 101 110 010
4	Isb = 0, no op	011 101	010 101 110 010
	Rshift Mplier	011 101	001 010 111 001
5	Prod = Prod + Mcand	011 101	100 111 111 001
	Rshift Mplier	011 101	010 011 111 100
6	Isb = 0, no op	011 101	010 011 111 100
	Rshift Mplier	011 101	001 001 111 110

3.4.3 Escreva um programa de linguagem assembly MIPS para calcular o produto de inteiros sem sinal A e B, usando a abordagem descrita na Figura 3.4.

Exercício 3.10

Em uma arquitetura de Von Neumann, grupos de bits não têm significados intrínsecos por si mesmos. O que um padrão de bits representa depende inteiramente de como é usado. A tabela a seguir mostra padrões de bits expressos em notação hexadecimal.

- a. 0x0C000000
- b. 0xC4630000

3.10.1 Qual número decimal o padrão de bits representa se for um inteiro em complemento de 2? E um inteiro sem sinal?

a.	201326592	201326592
b.	-1000144896	3294822400

3.10.2 Se este padrão de bits for colocado no Registrador de Instruções, qual instrução MIPS será executada?

a.	jal 0x00000000
b.	lwc1 \$3,0(\$3)

3.10.3 Qual o número decimal que o padrão de bits representa se for um número em ponto flutuante? Use o padrão IEEE 754.

a.	0x0C000000 = 0000 1100 0000 0000 0000 0000 0000 0000 = 0 0001 1000 0000 0000 0000 0000 0000 000 sign is positive exp = 0x18 = 24 - 127 = -103 there is a hidden 1 mantissa = 0 answer = 1.0×2^{-103}
b.	0xC4630000 = 1100 0100 0110 0011 0000 0000 0000 0000 = 1 1000 1000 1100 0110 0000 0000 0000 000 sign is negative exp = 0x88 = 136 - 127 = 9 there is a hidden 1 mantissa = 0xC60000 = $12 \times 16^{-1} + 6 \times 16^{-2}$ = .75 + .0234375 answer = -1.7734375×2^9

Exercício 3.11

No padrão de ponto flutuante IEEE 754, o expoente é armazenado no formato enviesado (também conhecido como “Excesso-N”). Essa abordagem foi selecionada porque queremos que um padrão tudo-zero seja o mais próximo de zero possível. Por causa do uso de um 1 implícito, se fôssemos representar o expoente no formato de complemento de dois, um padrão tudo-zero seria o número 1! (Lembre-se, qualquer coisa levantada para o poder zeroth é 1, então $1.00 = 1$.) Existem muitos outros aspectos do padrão IEEE 754 que existem para ajudar as unidades de ponto flutuante de hardware a trabalhar mais rapidamente. No entanto, em muitas máquinas mais antigas, os cálculos de ponto flutuante eram tratados em software e, portanto, outros formatos eram usados. Considere os números a seguir.

- a. -1.5625×10^{-1}
- b. 9.356875×10^2

3.11.1 Escreva o padrão de bits binário assumindo um formato semelhante ao empregado pela DEC PDP-8 (os 12 bits mais à esquerda são o expoente armazenado como um número em complemento de

dois e os 24 bits mais à direita são a mantissa armazenada como um número em complemento de dois). Não existe '1' implícito. Comente como o intervalo e a precisão desse padrão de 36 bits se comparam aos padrões IEEE 754 de precisão simples e dupla.

a.	$-1.5625 \times 10^{-1} = -.15625 \times 10^0$ $= -.00101 \times 2^0$ move the binary point 2 to the right $= -.101 \times 2^{-2}$ exponent = -2, mantissa = $-.101000000000000000000000$ answer: 11111111110101100000000000000000
b.	$9.356875 \times 10^2 = 935.6875 \times 10^0$ $= 0x3A7.B \times 16^0 = 1110100111.1011 \times 2^0$ move the binary point 10 to the left $= .11101001111011 \times 2^{10}$ exponent = +10, mantissa = $+.11101001111011$ answer: 000000001010011101001111011000000000

3.11.2 A NVIDIA tem um formato “metade”, que é semelhante ao IEEE 754, exceto pelo fato de ter apenas 16 bits de largura. O bit mais à esquerda ainda é o bit de sinal, o expoente tem 5 bits e é armazenado no formato em excesso de 56, e a mantissa tem 10 bits de comprimento. Assume-se o 1 implícito. Escreva o padrão de bits assumindo uma versão modificada desse formato, que usa um formato de excesso de 16 para armazenar o expoente. Comente como o intervalo e a precisão desse formato de ponto flutuante de 16 bits se comparam ao padrão IEEE 754 de precisão única.

a.	$-1.5625 \times 10^{-1} = -.15625 \times 10^0$ $= -.00101 \times 2^0$ move the binary point 3 to the right, $= -1.01 \times 2^{-3}$ exponent = -3 = $-3 + 16 = 13$, mantissa = $-.0100000000$ answer: 1011010100000000
b.	$9.356875 \times 10^2 = 935.6875 \times 10^0$ $= 0x3A7.B \times 16^0 = 1110100111.1011 \times 2^0$ move the binary point 9 to the left $= 1.1101001111011 \times 2^9$ exponent = +9 = $9 + 16 = 25$, mantissa = $+.1101001111011$ answer: 0110011101001111